

Ejercicios Prácticos - Semana 2 GitHub + SQL Avanzado + EDA Completo



Bootcamp: Fundamentos de Ingeniería de Datos

Tema: EDA (Exploratory Data Analysis) con SQL

Dataset: `bootcamp.bronze.properties_bronze` (datos crudos de propiedades inmobiliarias)

Instrucciones Generales

Preparación

1. Crear un nuevo notebook:
`Semana_2_EDA_Propiedades`
2. Asegurarte de tener el catálogo bootcamp y esquema landing creados
3. Verificar que la tabla
`bootcamp.bronze.properties_bronze` existe y tiene datos

Ejecución

1. Usar `%sql` al inicio de cada celda SQL
2. Documentar tus hallazgos en comentarios
3. Subir tus soluciones a GitHub



Importante: Un buen EDA es la base de un buen pipeline. Documenta todo lo que encuentres, ya que los hallazgos de EDA guían las transformaciones en Silver.

PARTE 0: Práctica de CTEs y Window Functions

Dataset: samples.nyctaxi.trips (tabla de ejemplo de Databricks)

Antes de comenzar con el EDA de propiedades, practiquemos con CTEs (Common Table Expressions) y Window Functions usando los datos de taxis de NYC que ya conoces de la Semana 1.

¿Qué son las CTEs?

Las CTEs (Common Table Expressions) son consultas temporales que mejoran la legibilidad y organización del código. Se definen con WITH y se usan cuando:

- Necesitas reutilizar una subconsulta múltiples veces
- Quieres hacer queries más legibles y organizadas
- Necesitas crear "vistas temporales" dentro de una query

¿Qué son las Window Functions?

Las Window Functions permiten realizar cálculos sobre un conjunto de filas relacionadas sin agrupar los datos. Son útiles para:

- Rankings y ordenamientos
- Comparar filas con promedios/totales generales
- Calcular acumulados y tendencias
- Análisis comparativos sin perder el detalle de cada fila

01	02	03
Ejercicio 0.1: CTE básica - Estadísticas por zona	Ejercicio 0.2: CTE múltiple - Análisis comparativo	Ejercicio 0.3: CTE para limpieza de datos
Objetivo: Usar una CTE para organizar cálculos.	Objetivo: Usar múltiples CTEs para organizar cálculos complejos.	Objetivo: Usar CTE para filtrar datos antes de analizar.
Pregunta: Calcula el promedio de tarifa y distancia por zona de inicio (pickup_zip). Usa una CTE para calcular primero las estadísticas por zona, y luego muestra solo las zonas con más de 100 viajes.	Pregunta: Compara el promedio de tarifa por hora del día con el promedio general. Usa dos CTEs: una para calcular el promedio por hora, otra para calcular el promedio general, y luego únelas para mostrar la diferencia.	Pregunta: Calcula estadísticas de distancia solo para viajes válidos (distancia > 0, tarifa > 0). Usa una CTE para filtrar primero los datos válidos, y luego calcula mínimo, máximo, promedio y mediana.
<i>Pista: WITH estadisticas_por_zona AS (...) luego filtra con HAVING</i>	<i>Pista: WITH promedio_por_hora AS (...), promedio_general AS (...)</i>	<i>Pista: WITH viajes_validos AS (SELECT ... WHERE ...)</i>
04	05	
Ejercicio 0.4: Window Function - ROW_NUMBER() para ranking	Ejercicio 0.5: Window Function - ROW_NUMBER() vs RANK() vs DENSE_RANK()	
Objetivo: Usar ROW_NUMBER() para crear rankings.	Objetivo: Comparar las 3 funciones de ranking sobre el mismo dataset y entender cómo manejan los empates.	
Pregunta: Crea un ranking de los viajes más caros. Muestra los top 20 viajes con su fecha, distancia, tarifa y el ranking (1 = más caro).	Pregunta: Usando la misma tabla de viajes, mostrá los top 20 viajes más caros con su fecha, tarifa y tres columnas de ranking: ROW_NUMBER(), RANK() y DENSE_RANK(). ¿Qué pasa cuando hay tarifas repetidas?	
<i>Pista: ROW_NUMBER() OVER (ORDER BY fare_amount DESC)</i>	<i>Pista: Usá las 3 funciones con OVER (ORDER BY fare_amount DESC) en el mismo SELECT</i>	

Ejercicios 0.6 - 0.10: Window Functions Avanzadas

1	Ejercicio 0.6: Comparar con promedio general Objetivo: Usar <code>AVG()</code> <code>OVER()</code> para comparar con el promedio. Pregunta: Para cada viaje, muestra la tarifa, la distancia, y cómo se compara la tarifa con el promedio general (diferencia y porcentaje de diferencia). <i>Pista: <code>AVG(fare_amount) OVER()</code> calcula el promedio de todas las filas</i>
2	Ejercicio 0.7: Promedio por partición Objetivo: Usar <code>PARTITION BY</code> para calcular promedios por grupo. Pregunta: Para cada viaje, muestra: fecha, tarifa, zona de inicio (<code>pickup_zip</code>), y el promedio de tarifa para esa misma zona (ej: si es zona 10001, muestra el promedio de todos los viajes de la zona 10001). <i>Pista: <code>AVG(fare_amount) OVER (PARTITION BY pickup_zip)</code></i>
3	Ejercicio 0.8: LAG() para comparar con anterior Objetivo: Usar <code>LAG()</code> para comparar con el registro anterior. Pregunta: Ordena los viajes por fecha y muestra: fecha, tarifa, y la tarifa del viaje anterior (si existe). Esto te permite ver si hay tendencias temporales. <i>Pista: <code>LAG(fare_amount) OVER (ORDER BY tpep_pickup_datetime)</code></i>
4	Ejercicio 0.9: SUM() acumulado Objetivo: Usar <code>SUM()</code> <code>OVER()</code> para calcular acumulados. Pregunta: Ordena los viajes por fecha y muestra: fecha, tarifa, y el total acumulado de tarifas hasta esa fecha (running total). <i>Pista: <code>SUM(fare_amount) OVER (ORDER BY tpep_pickup_datetime ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)</code></i>
5	Ejercicio 0.10: Combinando CTEs y Window Functions Objetivo: Combinar ambas técnicas en una query compleja. Pregunta: Usa CTEs para: 1. Filtrar viajes válidos (<code>distancia > 0</code>, <code>tarifa > 0</code>) 2. Calcular estadísticas por zona (promedio, máximo, mínimo) 3. Luego usa Window Functions para rankear las zonas por promedio de tarifa 4. Muestra el top 10 zonas más caras con su ranking <i>Pista: Combina <code>WITH</code> para las CTEs y <code>ROW_NUMBER()</code> <code>OVER()</code> para el ranking</i>



Setup del Entorno en Databricks

Vamos a preparar nuestro espacio de trabajo configurando las capas iniciales donde guardaremos nuestros datos crudos.

🚩 Paso 2: Crear el esquema Landing

Definimos nuestro esquema dentro del catálogo para organizar los datos en formato "Landing" o capa de archivo inicial.

```
%sql
-- =====
-- PASO 2: Crear el esquema Landing
-- =====

-- Crear catalogo si no existe
Create catalog bootcamp;
-- Usar tu catálogo
USE CATALOG bootcamp;

-- Crear esquema Landing
CREATE SCHEMA IF NOT EXISTS bootcamp.landing
COMMENT 'Esquema para archivos';

-- Crear esquema Raw
CREATE SCHEMA IF NOT EXISTS bootcamp.bronze
COMMENT 'Esquema para datos crudos sin procesar';

-- Verificar
SHOW SCHEMAS;
```

📦 Paso 3: Crear el Volume para archivos

Creamos un volumen gestionado por Databricks para almacenar de forma segura nuestros archivos CSV fuente.

```
%sql
-- =====
-- PASO 3: Crear the Volume para archivos
-- =====

-- Crear volume tipo MANAGED (Databricks administra el storage)
CREATE VOLUME IF NOT EXISTS bootcamp.landing.archivos
COMMENT 'Volume para almacenar archivos CSV crudos';

-- Verificar que se creó
SHOW VOLUMES IN bootcamp.landing;
```

📁 Paso 4: Subir el archivo CSV



Instrucciones de carga

- En el panel izquierdo de Databricks, hacé clic en **"Catalog"**.
- Navegá hasta tu catálogo → `landing` → `archivos`.
- Hacé clic en el botón **"Upload"** y arrastrá el archivo `properties\_raw.csv`.

📍 **Ruta del archivo:** /Volumes/bootcamp/landing/archivos/properties_raw.csv

✅ Verificamos que el archivo esté disponible en el volumen antes de procesarlo.

```
%sql
-- =====
-- PASO 4: Verificar que el archivo se subió correctamente
-- =====

-- Listar archivos en el volume
LIST '/Volumes/bootcamp/landing/archivos/';
```

🔨 Paso 5: Crear la tabla properties_bronze

Finalmente, transformamos los datos crudos en una tabla relacional, filtrando registros con URLs válidas para asegurar la calidad.

```
%sql
-- Leer directo del archivo es posible, y es útil también para analizarlo

SELECT * FROM read_files(
  '/Volumes/bootcamp/landing/archivos/properties_raw.csv',
  format => 'csv',
  header => true
)

-- PASO 5: Crear la tabla properties_bronze
-- =====

-- Primero, eliminamos la tabla si existe (para poder recrearla)
DROP TABLE IF EXISTS bootcamp.bronze.properties_bronze;

-- Crear tabla EXTERNA leyendo el CSV y nos quedamos solo con los registros que tienen url válida
CREATE TABLE bootcamp.bronze.properties_bronze
SELECT * FROM read_files(
  '/Volumes/bootcamp/landing/archivos/properties_raw.csv',
  format => 'csv',
  header => true
)
where url like 'https%' --Filtramos por aquellos links que sean consistentes
;
```

🔍 Comprobación final de la tabla creada:

```
%sql
-- Verificar que la tabla se creó correctamente
SHOW TABLES IN bootcamp.bronze;
```

PARTE 1-3: Exploración Inicial y Análisis de Calidad



PARTE 1: Exploración Inicial

- Ejercicio 1.1:** Verificar datos disponibles - ¿Cuántos registros hay en total en la tabla `bootcamp.bronze.properties_bronze`? *Pista: Usa `COUNT(*)`*
- Ejercicio 1.2:** Explorar estructura - ¿Qué columnas tiene la tabla? ¿Qué tipos de datos tienen? *Pista: Usa `DESCRIBE TABLE` o `SELECT * LIMIT 0`*
- Ejercicio 1.3:** Ver muestra de datos - Muestra las primeras 10 filas del dataset, pero solo las columnas más importantes: `id`, `ubicacion`, `precio`, `expensas`, `tipo_de_operacion`, `moneda`, `ambientes`, `metros_cuadrados_totales`, `antiguedad`, `estado`, `zona`. *Pista: Usa `SELECT` con columnas específicas y `LIMIT 10`*



PARTE 2: Análisis de Valores Nulos

- Ejercicio 2.1:** Contar nulos por columna - ¿Cuántos valores nulos hay en cada columna importante? Calcula para: `precio`, `expensas`, `tipo_de_operacion`, `moneda`, `ambientes`, `metros_cuadrados_totales`, `metros_cuadrados_cubiertos`, `orientacion_cardinal`, `piso`, `cochera`, `antiguedad`, `estado`, `zona`. *Pista: Usa `COUNT(*)` - `COUNT(columna)` para cada columna. Puedes usar CTEs para organizar.*
- Ejercicio 2.2:** Porcentaje de nulos - ¿Qué porcentaje de registros tienen valores nulos en las columnas clave? Calcula el porcentaje para: `precio`, `expensas`, `ambientes`, `metros_cuadrados_totales`, `metros_cuadrados_cubiertos`, `orientacion_cardinal`, `antiguedad`. *Pista: Usa `ROUND((total - COUNT(columna)) * 100.0 / total, 2)`*
- Ejercicio 2.3:** Identificar columnas críticas - ¿Qué columnas tienen más del 50% de valores nulos? ¿Cuáles son críticas para el análisis? *Pista: Usa el resultado del ejercicio anterior y filtra con `HAVING` o `WHERE`*



PARTE 3: Cardinalidad y Distribución

- Ejercicio 3.1:** Distribución de tipo de operación - ¿Qué valores únicos hay en `tipo_de_operacion`? ¿Cuántos registros hay de cada tipo? ¿Qué porcentaje representa cada uno? *Pista: Usa `GROUP BY`, `COUNT(*)`, y `SUM(COUNT(*) OVER())` para calcular porcentajes*
- Ejercicio 3.2:** Distribución de moneda - ¿Qué monedas hay en el dataset? ¿Cuántos registros hay de cada una? ¿Hay alguna moneda que no esperabas? *Pista: `GROUP BY moneda` con conteo y porcentaje*
- Ejercicio 3.3:** Distribución de ambientes - ¿Cuántas propiedades hay por cantidad de ambientes? ¿Cuál es la distribución? Ordena por cantidad de ambientes. *Pista: `GROUP BY ambientes ORDER BY ambientes`*
- Ejercicio 3.4:** Top zonas - ¿Cuáles son las 15 zonas con más propiedades? Muestra zona, cantidad y porcentaje del total. *Pista: `GROUP BY zona, COUNT(*)`, porcentaje, `ORDER BY cantidad DESC LIMIT 15`*
- Ejercicio 3.5:** Distribución de estado - ¿Qué valores hay en la columna `estado`? ¿Cuántas propiedades hay de cada estado? *Pista: `GROUP BY estado ORDER BY cantidad DESC`*

PARTE 4: Estadísticas Descriptivas de Variables Numéricas

1

Ejercicio 4.1: Estadísticas de precio por moneda y tipo de operación

Objetivo: Analizar precios separados por moneda.

Pregunta: Para cada moneda y tipo de operación, calculá: cantidad de registros, precio mínimo, máximo, promedio, mediana, percentil 25 y percentil 75. Solo incluye precios válidos (mayores a 0).

Pista: Usa MIN(), MAX(), AVG(), PERCENTILE() con GROUP BY moneda



Nota: Si los datos tienen problemas de formato (como strings en vez de números), primero necesitarás limpiarlos. Revisa el notebook de solución para ver cómo se hace.

2

Ejercicio 4.2: Estadísticas de metros cuadrados

Objetivo: Analizar las superficies de las propiedades.

Pregunta: Calcula estadísticas (mínimo, máximo, promedio, mediana) para metros_cuadrados_totales y metros_cuadrados_cubiertos por separado. Solo incluye valores válidos (mayores a 0).

Pista: Puedes usar UNION ALL para mostrar ambas columnas en un solo resultado

3

Ejercicio 4.3: Análisis de antigüedad

Objetivo: Identificar valores placeholder en antigüedad.

Pregunta: ¿Qué valores únicos hay en la columna antigüedad? ¿Cuántos registros tienen cada valor? Ordena por cantidad descendente y muestra los top 20.

Pista: GROUP BY antigüedad ORDER BY cantidad DESC LIMIT 20

Pregunta de reflexión: ¿Notas algún valor que parece ser un placeholder (como 999)?

4

Ejercicio 4.4: Estadísticas de antigüedad sin placeholders

Objetivo: Calcular estadísticas reales excluyendo valores placeholder.

Pregunta: Calcula estadísticas de antigüedad EXCLUYENDO el valor 999 (que parece ser un placeholder). Muestra: cantidad, mínimo, máximo, promedio, mediana.

Pista: Usa WHERE antigüedad != 999 AND antigüedad IS NOT NULL AND antigüedad >= 0

PARTE 5: Detección de Problemas de Calidad



Ejercicio 5.1

Reporte de calidad usando CTEs

Crea un reporte que muestre: Total de registros, Cantidad de precios inválidos (NULL o ≤ 0), Cantidad de metros cuadrados inválidos (NULL o ≤ 0), Cantidad de antigüedad con valor 999 (placeholder), Cantidad de ambientes inválidos (NULL o 0), Cantidad de moneda vacía, Cantidad de tipo_operacion vacía, Porcentaje de cada problema sobre el total.

Pista: Usa CTEs para organizar. Calcula totales y problemas por separado, luego únelos.



Ejercicio 5.2

Detectar duplicados

¿Hay registros duplicados? Agrupa por precio y url y encuentra grupos que aparecen más de una vez.

Muestra: cantidad de grupos duplicados, total de registros duplicados, y cuántos registros "extra" hay por duplicación.

Pista: GROUP BY precio, url HAVING COUNT() > 1, luego suma los resultados*



Ejercicio 5.3

Ver ejemplos de duplicados

Muestra ejemplos de duplicados. Para cada grupo duplicado (precio + url), muestra cuántas veces aparece. Ordena por cantidad de apariciones descendente y limita a 10.

Pista: GROUP BY precio, url HAVING COUNT() > 1 ORDER BY COUNT(*) DESC LIMIT 10*



Ejercicio 5.4

Detectar outliers en precio

Calcula los percentiles 1 y 99 de precio por moneda. Luego identifica propiedades con precios fuera de ese rango (outliers extremos).

Pista: Usa CTE para calcular percentiles, luego filtra registros fuera del rango

PARTE 6-7: Análisis Avanzado y Documentación

PARTE 6: Análisis Avanzado con Window Functions

Ejercicio 6.1: Ranking de zonas por precio - Crea un ranking de zonas por precio promedio. Muestra: zona, precio promedio, cantidad de propiedades, y el ranking (1 = más cara). *Pista: Calcula precio promedio por zona, luego usa ROW_NUMBER() OVER (ORDER BY precio_promedio DESC)*

Ejercicio 6.2: Comparar con promedio general - Para cada zona, muestra: zona, precio promedio de la zona, y cómo se compara con el precio promedio general (diferencia y porcentaje). *Pista: Usa AVG(precio) OVER() para calcular el promedio general*

Ejercicio 6.3: Análisis temporal (si hay fecha) - Si hay una columna de fecha, agrupa por mes y muestra: mes, cantidad de propiedades, precio promedio. ¿Hay alguna tendencia? *Pista: Usa DATE_TRUNC('month', fecha) o YEAR() y MONTH() para agrupar*

PARTE 7: Documentación de Hallazgos

Ejercicio 7.1: Resumen ejecutivo - Crea un resumen que documente:

1. Total de registros analizados
2. Principales problemas de calidad encontrados
3. Porcentaje de datos válidos vs inválidos
4. Recomendaciones para la limpieza en Silver

Pista: Combina los resultados de ejercicios anteriores en un resumen estructurado

Checklist de Finalización

- Practiqué CTEs y Window Functions (Parte 0)
- Completé la exploración inicial (Parte 1)
- Analicé valores nulos (Parte 2)
- Analicé distribuciones categóricas (Parte 3)
- Calculé estadísticas descriptivas (Parte 4)
- Identifiqué problemas de calidad (Parte 5)
- Apliqué Window Functions (Parte 6)
- Documenté mis hallazgos (Parte 7)
- Subí mi notebook a GitHub
- Creé un README explicando los hallazgos principales



EDA es iterativo

Empieza con lo básico y profundiza según lo que encuentres



Documenta todo

Los hallazgos de EDA guían las transformaciones en Silver



Usa CTEs

Organizan queries complejas y las hacen más legibles



Window Functions

Son poderosas para análisis comparativos



Valores placeholder

Identifica valores como 999, -1, 'N/A' que representan "no disponible"



Nota sobre Formato de Datos: Si encuentras que los datos tienen problemas de formato (strings donde deberían ser números), revisa el notebook de solución para ver cómo se crea la vista temporal propiedades_clean que limpia los tipos antes del análisis.

¡Un buen EDA es la base de un buen pipeline!